

LLMs y OpenFrameworks

DE LOS DATOS A LA EXPERIENCIA



Usamos un modelo sin dejar de cuestionarlo

Ya vimos: tokens · embeddings · atención · pesos · datos y sesgos

ANTES · “NO FUE EL PROMPT”

Analizar y evaluar

Exploramos datos y representaciones.
Comparamos resultados para observar sesgos.

AHORA · OPENFRAMEWORKS

Consumir y diseñar

Usamos el modelo **Qwen** ya entrenado (multilenguaje). Validamos su respuesta y decidimos qué hace en nuestra experiencia.

Cambian la tarea y el modelo. Conservamos la mirada crítica.

Hacemos inferencia: no ajustamos los pesos del modelo.

Ustedes construyen el entorno, el addon conecta

SU PROYECTO EN OF

Interfaz, escena, interacción y lógica propias

ofxLocalLLM

Envía pedidos HTTP y entrega las respuestas al sketch. No trae una interfaz automática.

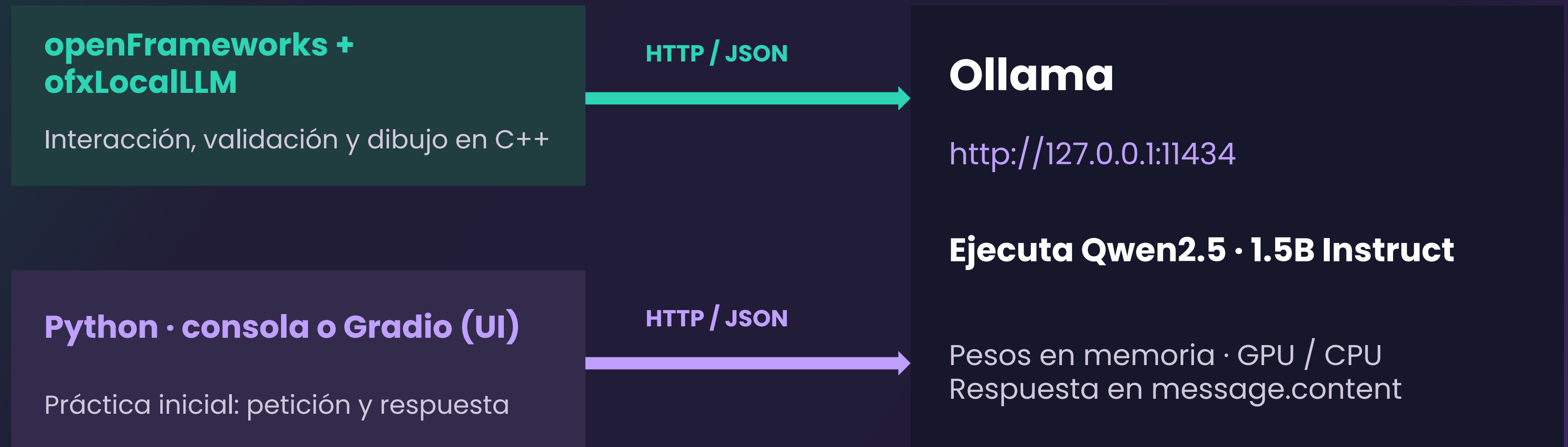
Ollama ejecuta Qwen en la computadora.

QUÉ DECIDEN USTEDES

- 01** Qué información enviar
- 02** Cuándo consultar al modelo
- 03** Cómo usar su respuesta
- 04** Qué hacer si falla

Python y OF consultan directamente a Ollama

Dos clientes independientes. Cada alumno ejecuta todo localmente.



OF funciona aunque la práctica Python esté cerrada.

Un mismo modelo, dos formas de interactuar



Captura: espera simulada · afiche grabado, sin inferencia

CHAT

Preguntar, responder y repreguntar con contexto.

AFICHE

Interpretar un texto en título, ánimo, ritmo y paleta.

El dibujo sigue activo mientras esperamos al LLM.

La respuesta es texto, el contrato permite usarla

AFICHE · cinco campos visuales

```
{
  "titulo": "Un mar de palabras",
  "animo": "calma",
  "ritmo": "lento",
  "paleta": "mar",
  "motivo": "Movimiento suave."
}
```

PosterSpec.h valida categorías y límites.
ChatSession.h agrega respuesta e historial.

Esc → E copia el diagnóstico para investigar el error.

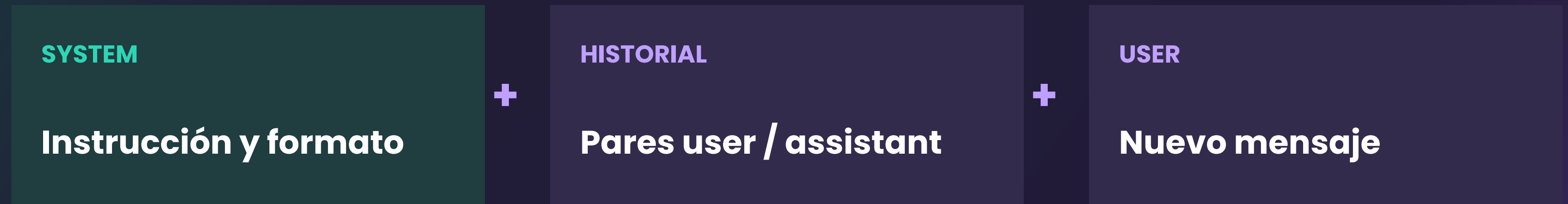
CHAT · respuesta + visual

```
{
  "respuesta": "Imaginemos un mar de palabras.",
  "visual": {
    "titulo": "Un mar de palabras",
    "animo": "calma", "ritmo": "lento",
    "paleta": "mar", "motivo": "Flujo suave."
  }
}
```

JSON válido ≠ interpretación correcta.
Ante un rechazo, conservamos la escena.

La memoria se reenvía, los pesos no cambian

Cada consulta vuelve a incluir los mensajes anteriores.



4 intercambios completos como máximo

Se guardan en RAM durante la ejecución. NUEVO CHAT borra el historial.

Probá: "Me llamo Noelia" → "¿Cómo me llamo?" → NUEVO CHAT → repetí la pregunta.

El LLM propone y OF decide cómo se mueve

Una interpretación del texto, no una medición de emociones ni de audio.

ANIMO

MOVIMIENTO EN OF

calma

Ondas que respiran suavemente

alegria

Círculos que rebotan

tension

Línea en zigzag

ritmo → velocidad paleta → colores

Mientras espera: "Pensando..." + puntos animados + tiempo transcurrido.

Construyan una experiencia que use la respuesta

01

Diseñar la interacción

Elegir entrada, evento y escena. Partir del ejemplo conectado.

02

Transformar los datos

Validar la respuesta y mapearla a una acción visual o sonora propia.

03

Probar y cuestionar

Comparar dos pedidos. Registrar una respuesta útil y otra problemática.

¿Qué decidió el modelo y qué decidieron ustedes?

Para empezar: cambiar una paleta o un movimiento en PosterView.h.

El script prepara Python, Ollama y el modelo

Guardar la carpeta completa de la clase en una ubicación definitiva.

macOS

```
bash install_env_mac.command
```

Linux

```
bash install_env_linux.sh
```

Windows

```
install_env_windows.cmd · doble clic
```

Al terminar: Python preparado, Ollama activo y Qwen precargado.

Primera instalación: internet y permisos del sistema. OF y su compilador se instalan aparte.

Ollama elige dónde cargar el modelo

La misma consulta de OF funciona en equipos distintos.

GPU compatible

Aceleración según hardware y memoria.
Puede combinar GPU y CPU.

Sin GPU compatible

La inferencia se ejecuta en CPU.
OF usa la misma conexión.

5 min

sin uso: puede liberarse la memoria.
La siguiente consulta vuelve a cargar el modelo.

Tras reiniciar: abrir Ollama o repetir el instalador.

Importar el ejemplo evita compilar una plantilla vacía

01 Copiar las carpetas completas

ofxLocalLLM → OF/addons/
llmPoster → OF/apps/myApps/

```
llmPoster/  
src/      código del ejemplo  
bin/data/ prompts y configuración  
addons.make
```

02 import → seleccionar llmPoster

Project path **OF/apps/myApps**

Project name **llmPoster**

Addons **ofxLocalLLM**

Additional source paths: vacío
Plataforma del equipo → Update

La ruta termina en myApps. Project Generator agrega el nombre llmPoster.

Compilar, conversar y observar qué cambia

COMPILAR · macOS / Linux con Make

```
cd RUTA_A_OF/apps/myApps/llmPoster  
make Release -j4  
make RunRelease
```

Windows: abrir y compilar el proyecto con el IDE correspondiente a su SDK.

Si aparece gris: revisar src/ofApp.cpp y la carpeta del proyecto que se compiló.

1 Abrir con Ollama activo

Debe verse la interfaz y el afiche grabado.

2 CHAT → escribir → ENVIAR

Aparece “Pensando...” durante la consulta.

3 Leer, repreguntar y comparar

Observar respuesta, tono y movimiento.

L: comprobar conexión · Esc → E: diagnóstico
Guía completa: README.md de la clase